Laporan Tugas Kecil

Strategi Algoritma (IF2211)

Disusun Oleh

Kelompok:

Muhammad Daffa Arrizki Yanma (13524133)

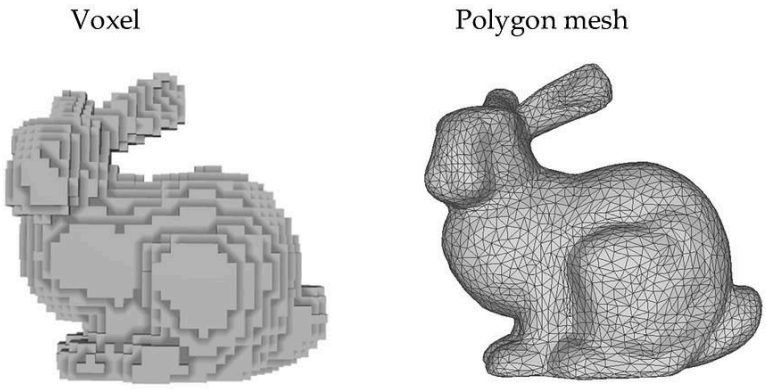
Muhammad Rafi Akbar (13524125)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

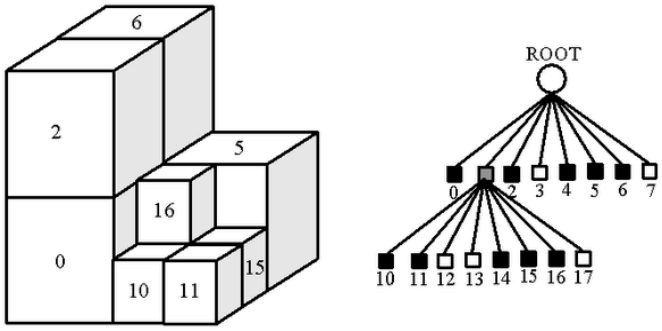
BANDUNG 2025

BAB I
PENDAHULUAN



Gambar 1 Visualisasi *voxelization*
Sumber: https://www.mdpi.com/1424-8220/22/18/7040

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data **Octree**, yaitu sebuah *tree* yang setiap percabangan terdiri dari 8 anak. Melalui *Octree*, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

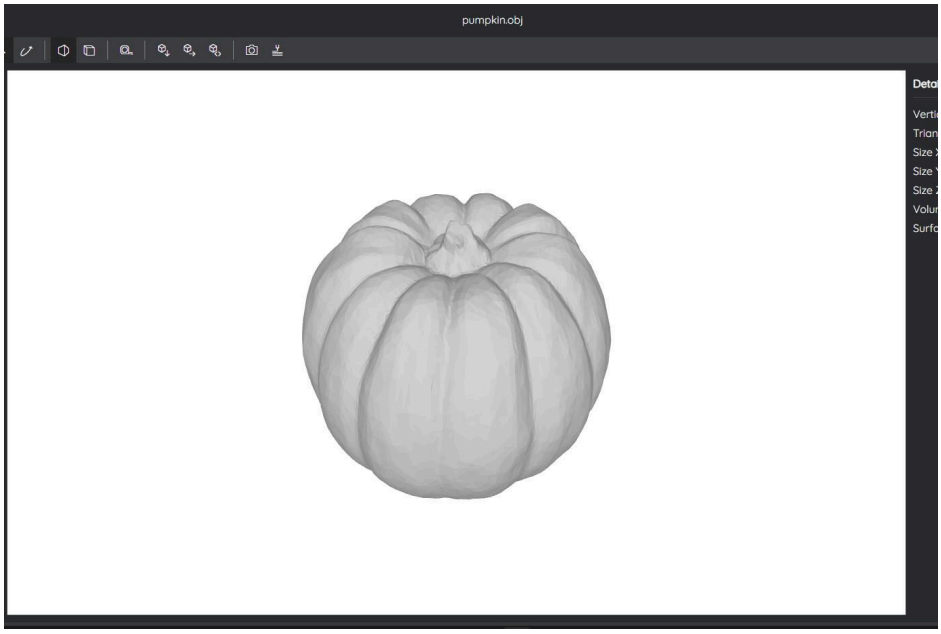


Gambar 2 Visualisasi *Octree* terhadap 3D Model
Sumber:
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

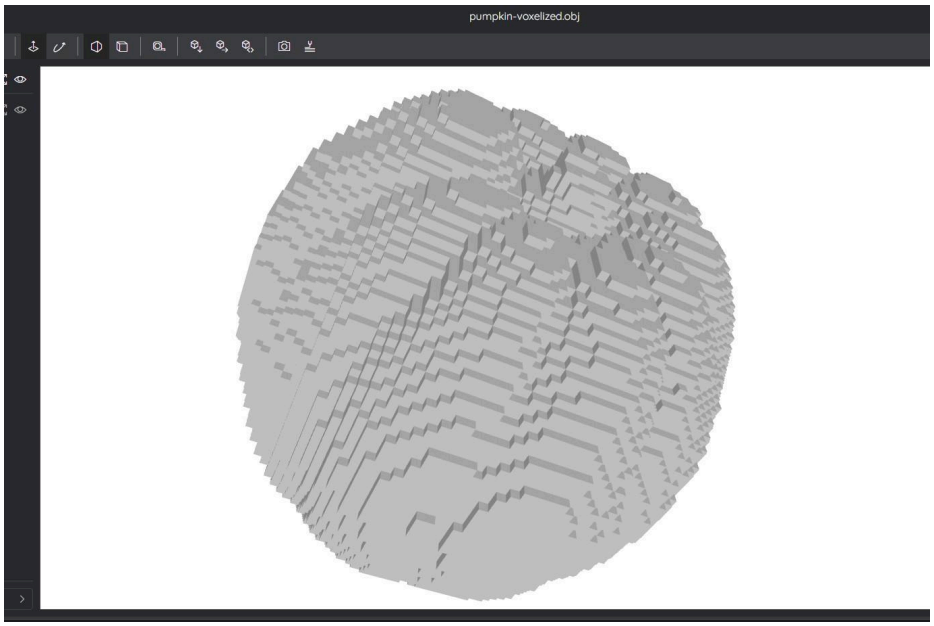
Tugas Anda adalah mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari *voxel* dengan memanfaatkan properti ***Octree***. Program melakukan konversi model dengan menggunakan algoritma ***divide and conquer.***

Ilustrasi Kasus

Diberikan sebuah model 3D sebagai berikut.



Di bawah adalah hasil perubahan.



BAB II
IMPLEMENTASI

2.1 Strategi Divide & Conquer

a. Divide

Fase divide dimulai dari bounding box awal yang melingkupi seluruh model 3D. Bounding box direpresentasikan sebagai kubus dengan koordinat minimum dan maksimum. Pada setiap iterasi, bounding box dibagi menjadi 8 octant (anak) dengan cara:

- Menghitung titik tengah pada sumbu X, Y, dan Z
- Membagi dengan hyperplane melalui titik tengah untuk menghasilkan 8 kubus anak dengan ukuran sama
- Masing-masing octant diindeks dari 0 hingga 7 menggunakan bit-flag berdasarkan posisi relatifnya terhadap titik tengah

Proses pembagian dilakukan secara rekursif hingga mencapai kedalaman maksimum yang ditentukan atau kondisi terminasi lainnya terpenuhi. Strategi ini menghasilkan struktur octree yang setiap node memiliki 8 anak, membentuk hierarchical spatial decomposition.

b. Conquer

Untuk setiap node dalam octree, dilakukan langkah-langkah berikut:

- Filtering Triangle

  Dari seluruh triangle dalam parent node, hanya triangle yang bersinggungan (touching/intersecting) dengan octant tersebut yang dimasukkan ke child node. Proses filtering menggunakan dua tahap:

  - Pemeriksaan menggunakan Axis-Aligned Bounding Box (AABB) collision detection
  - Validasi menggunakan Separating Axis Theorem (SAT)

- Base Cases

  Recursion berhenti pada dua kondisi:

  - Depth Maksimum: Jika kedalaman node telah mencapai maxDepth, node dinyatakan sebagai voxel (leaf node) dan tidak perlu subdivisi lebih lanjut. Voxel ini akan dioutput sebagai unit 3D dalam representasi mesh akhir.

- Node Kosong: Jika tidak ada triangle yang bersinggungan dengan octant tersebut, node tidak memiliki kontribusi pada surface model dan dapat di-skip tanpa subdivisi. Node ini tidak akan menghasilkan voxel output.

- Recursive Division
  Untuk node yang belum memenuhi base case, proses dilanjutkan dengan:

  - Membuat 8 child node (octant)
  - Melakukan distribusi triangle dari parent ke masing-masing child berdasarkan hasil filtering
  - Memanggil recursively untuk memproses setiap child dengan parameter kedalaman yang bertambah

c. Combine

Setelah octree selesai dibangun, voxel dikumpulkan melalui depth-first traversal. Untuk setiap node yang berstatus *isSurface* (voxel), dilakukan:

- Konversi koordinat bounding box node menjadi 8 vertices (corner points dari kubus)
- Pembuatan 12 triangle faces untuk merepresentasikan seluruh permukaan kubus
- Penulisan vertices dan faces ke file output dalam format OBJ

2.2 Implementasi Kode

a. Struktur vektor3d

Struktur ini merepresentasikan titik atau vektor 3D. Di dalamnya juga ada operasi dasar seperti pengurangan vektor, cross product, dan dot product.

```
struct vektor3d {
    float x, y, z;
    vektor3d(float x = 0, float y = 0, float z = 0) : x(x), y(y), z(z) {}

    // inisialisasi x y z
    vektor3d operator-(const vektor3d& o) const {
        return {
            x - o.x, y - o.y, z - o.z
        };
    }
    // cross product
    vektor3d cross(const vektor3d& o) const {
```

```
        return {
            y * o.z - z * o.y,
            z * o.x - x * o.z,
            x * o.y - y * o.x
        };
    }
    // dot operator
    float dotop(const vektor3d& o) const {
        return x * o.x + y * o.y + z * o.z;
    }
};
```

b. Struktur triangle

Struktur ini menyimpan satu segitiga yang terdiri dari tiga titik. Data ini dipakai sebagai representasi face hasil parsing file OBJ

```
struct triangle {
    vektor3d v0, v1, v2;
};
```

c. Struktur AABB

Struktur AABB digunakan untuk menyimpan bounding box dengan titik minimum dan maksimum. Method center() dipakai untuk menghitung titik tengah box saat proses pembagian octree.

```
// pembatas
struct AABB {
    vektor3d min, max;
    vektor3d center() const {
        return {
            (min.x + max.x) / 2,
            (min.y + max.y) / 2,
            (min.z + max.z) / 2
        };
    }
};
```

d. Struktur nodeOctree

Struktur ini merepresentasikan satu node octree. Setiap node menyimpan bounding box, daftar indeks triangle, delapan child, kedalaman node, dan penanda apakah node tersebut merupakan voxel permukaan.

```
struct nodeOctree {
    AABB bounds;
    std::vector<int> triangles;
    // subdivisi 8 child
    nodeOctree* children[8];
    int depth;
    bool isSurface;
    // inisialisasi node
    nodeOctree(AABB b, int d) : bounds(b), depth(d), isSurface(false) {
        for (int i = 0; i < 8; i++) children[i] = nullptr;
    }
};
```

e. Fungsi parserindexface

Fungsi ini mengambil indeks vertex dari token face OBJ. Jika token berbentuk 1/2/3, fungsi hanya mengambil bagian sebelum /, lalu mengubahnya menjadi integer.

```
bool parserindexface(const std::string& input, int& result) {
    if (input == "") return false;
    // cari index vertex
    size_t slashIndex = input.find('/');
    std::string numberText;

    //not found
    if (slashIndex == std::string::npos) {
        numberText = input;
    }

    else {
        numberText = input.substr(0, slashIndex);
    }

    if (numberText == "") return false;
```

```
    try {
        result = std::stoi(numberText);
        return true;
    } catch (const std::exception&) {
        return false;
    }

    return true;
}
```

f. Fungsi objParser

Fungsi ini membaca file OBJ. Baris v disimpan sebagai vertex, sedangkan baris f dibaca sebagai face segitiga lalu dikonversi menjadi objek triangle. Fungsi juga memvalidasi format file agar hanya menerima vertex 3D dan face segitiga.

```
//obj parser

bool objParser(const std::string& path, std::vector<vektor3d>& vertices, std::vector<triangle>&
triangles) {
    std::ifstream file(path);

    if (!file.is_open()) {
        std::cerr << "Error: tidak bisa membuka file " << path << "\n";
        return false;
    }

    std::string line;
    bool hasVertex = false, hasFace = false;
    while (std::getline(file, line)) {
        if (line.empty()) continue;

        std::istringstream ss(line);
        std::string token;
        ss >> token;

        // token v = vertex
        // token f = face
        if (token == "v") {
            float x, y, z;
```

```
        if (!(ss >> x >> y >> z)) {
            std::cerr << "Error: format vertex tidak valid" << line << "\n";
            return false;
        }
        // Cek tidak boleh ada extra koordinat
        std::string extra;
        if (ss >> extra) {
            std::cerr << "Error: vertex harus 3 koordinat (v x y z): " << line << "\n";
            return false;
        }
        vertices.push_back({x, y, z});
        hasVertex = true;

    } else if (token == "f") {
        int idx[3];
        // Baca 3 vertices
        for (int i = 0; i < 3; i++) {
            std::string part;
            if (!(ss >> part)) {
                std::cerr << "Error: face harus 3 vertices (f i j k): " << line << "\n";
                return false;
            }
            if (!parserindexface(part, idx[i])) {
                std::cerr << "Error: format face tidak valid (harus: f i j k): " << line << "\n";
                return false;
            }
            if (idx[i] < 1 || idx[i] > static_cast<int>(vertices.size())) {
                std::cerr << "Error: indeks face di luar jangkauan: " << idx[i] << "\n";
                return false;
            }
            idx[i]--;
        }
        // Cek tidak boleh ada extra vertices (hanya triangular)
        std::string extra;
        if (ss >> extra) {
            std::cerr << "Error: face harus 3 vertices (f i j k): " << line << "\n";
            return false;
        }
```

```
            triangles.push_back({vertices[idx[0]], vertices[idx[1]], vertices[idx[2]]});
            hasFace = true;
        }



    }
    if (!hasVertex || !hasFace) {
        std::cerr << "Error: file .obj tidak valid (harus punya baris v dan f)\n";
        return false;
    }
    return true;

}
```

g. Fungsi getBoundBox

Fungsi ini menghitung bounding box awal dari seluruh vertex model. Setelah itu, box dipaksa menjadi kubus dengan panjang sisi sama, lalu ditambah sedikit padding agar model tetap aman berada di dalam box.

```
AABB getBoundBox(const std::vector<vektor3d>& vertices) {
    AABB box;
    box.min = vertices[0];
    box.max = vertices[0];

    for (const vektor3d& v : vertices) {
        box.min.x = std::min(box.min.x, v.x);
        box.min.y = std::min(box.min.y, v.y);
        box.min.z = std::min(box.min.z, v.z);

        box.max.x = std::max(box.max.x, v.x);
        box.max.y = std::max(box.max.y, v.y);
        box.max.z = std::max(box.max.z, v.z);
    }

    float dx = box.max.x - box.min.x;
    float dy = box.max.y - box.min.y;
    float dz = box.max.z - box.min.z;
    float maxsize = std::max(std::max(dx, dy), dz);
```

```
    box.max.x = box.min.x + maxsize;
    box.max.y = box.min.y + maxsize;
    box.max.z = box.min.z + maxsize;

    float padding = maxsize * 0.001f;

    box.min.x -= padding;
    box.min.y -= padding;
    box.min.z -= padding;

    box.max.x += padding;
    box.max.y += padding;
    box.max.z += padding;

    return box;
}
```

h. Fungsi quickBoxCheck

Fungsi ini melakukan pengecekan awal apakah triangle mungkin bersinggungan dengan box. Jika rentang koordinat triangle tidak overlap dengan box pada salah satu sumbu, triangle langsung ditolak.

```
bool quickBoxCheck(const triangle& tri, const AABB& box) {
    float minX = std::min({tri.v0.x, tri.v1.x, tri.v2.x});
    float minY = std::min({tri.v0.y, tri.v1.y, tri.v2.y});
    float minZ = std::min({tri.v0.z, tri.v1.z, tri.v2.z});

    float maxX = std::max({tri.v0.x, tri.v1.x, tri.v2.x});
    float maxY = std::max({tri.v0.y, tri.v1.y, tri.v2.y});
    float maxZ = std::max({tri.v0.z, tri.v1.z, tri.v2.z});

    if (maxX < box.min.x || minX > box.max.x) {
        return false;
    }
    if (maxY < box.min.y || minY > box.max.y) {
        return false;
```

```
    }
    if (maxZ < box.min.z || minZ > box.max.z) {
        return false;
    }

    return true;
}
```

i. **Fungsi detailedCheck**

Fungsi ini melakukan pengecekan lebih detail setelah quick check. Triangle dipindahkan ke koordinat lokal box, lalu diuji overlap terhadap sumbu utama dan bidang triangle. Jika lolos, triangle dianggap benar-benar menyentuh box.

```
bool detailedCheck(const triangle& tri, const AABB& box) {
    vektor3d center = box.center();

    float hx = (box.max.x - box.min.x) * 0.5f;
    float hy = (box.max.y - box.min.y) * 0.5f;
    float hz = (box.max.z - box.min.z) * 0.5f;

    vektor3d a = tri.v0 - center;
    vektor3d b = tri.v1 - center;
    vektor3d c = tri.v2 - center;

    vektor3d ab = b - a;
    vektor3d ac = c - a;

    // cek overlap sumbu utama
    if (std::max({a.x, b.x, c.x}) < -hx || std::min({a.x, b.x, c.x}) > hx) {
        return false;
    }
    if (std::max({a.y, b.y, c.y}) < -hy || std::min({a.y, b.y, c.y}) > hy) {
        return false;
    }
    if (std::max({a.z, b.z, c.z}) < -hz || std::min({a.z, b.z, c.z}) > hz) {
        return false;
    }
```

```
    // cek plane triangle
    vektor3d normal = ab.cross(ac);
    float r = hx * std::abs(normal.x) + hy * std::abs(normal.y) + hz * std::abs(normal.z);

    if (std::abs(normal.dotop(a)) > r) return false;



    return true;
}
```

j. Fungsi isTriangleTouching

Fungsi ini menggabungkan dua tahap pengecekan. Pertama dilakukan quick check, lalu jika masih mungkin overlap, dilanjutkan dengan detailed check.

```
bool isTriangleTouching(const triangle&tri, const AABB& box) {

    if (!quickBoxCheck(tri, box)) return false;
    // double check

    return detailedCheck(tri, box);

}
```

k. Fungsi splitBox

Fungsi ini membagi satu bounding box parent menjadi salah satu dari delapan child box. Penentuan posisi child dilakukan dengan bit-flag pada childIndex untuk sumbu X, Y, dan Z.

```
// bagi box menjadi 8
AABB splitBox(const AABB& parent, int childIndex) {
    vektor3d mid = parent.center();
    AABB result;

    //  range X
    if (childIndex & 1) {
```

```
        result.min.x = mid.x;
        result.max.x = parent.max.x;
    } else {
        result.min.x = parent.min.x;
        result.max.x = mid.x;
    }

    // Y
    if (childIndex & 2) {
        result.min.y = mid.y;
        result.max.y = parent.max.y;
    } else {
        result.min.y = parent.min.y;
        result.max.y = mid.y;
    }

    // Z
    if (childIndex & 4) {
        result.min.z = mid.z;
        result.max.z = parent.max.z;
    } else {
        result.min.z = parent.min.z;
        result.max.z = mid.z;
    }

    return result;
}
```

l. Fungsi buildTree

Fungsi ini adalah inti pembentukan octree. Jika node kosong, proses dihentikan. Jika kedalaman maksimum tercapai, node ditandai sebagai voxel permukaan. Jika belum, node dibagi menjadi delapan child, lalu triangle parent disaring ke masing-masing child dan diproses rekursif.

```
// octree rekursif
void buildTree(nodeOctree* node,
               const std::vector<triangle>& tris,
```

```
            int maxDepth) {

    nodesPerDepth[node->depth]++;

    if (node->triangles.empty()) {
        skippedPerDepth[node->depth]++;
        return;
    }

    if (node->depth >= maxDepth) {
        node->isSurface = true;
        totalVoxels++;
        return;
    }

    // Bagi jadi 8 bagian
    for (int i = 0; i < 8; i++) {
        AABB childBox = splitBox(node->bounds, i);
        nodeOctree* child = new nodeOctree(childBox, node->depth + 1);
        node->children[i] = child;

        for (int t : node->triangles) {
            if (isTriangleTouching(tris[t], childBox)) {
                child->triangles.push_back(t);
            }
        }

        // Rekursi ke child
        buildTree(child, tris, maxDepth);
    }

    node->triangles.clear();
}
```

m. Fungsi writeVoxels

Fungsi ini menelusuri octree dan menulis node leaf yang bertanda isSurface sebagai voxel ke file OBJ. Setiap voxel diubah menjadi 8 vertex dan 12 face segitiga.

```
// Traversal tree dan tulis voxel
void writeVoxels(nodeOctree* node, std::ofstream& out, int& offset) {
    if (!node) return;

    // Kalau voxel
    if (node->isSurface) {
        vektor3d mn = node->bounds.min;
        vektor3d mx = node->bounds.max;

        std::array<vektor3d, 8> pts = {{
            {mn.x, mn.y, mn.z}, {mx.x, mn.y, mn.z},
            {mx.x, mx.y, mn.z}, {mn.x, mx.y, mn.z},
            {mn.x, mn.y, mx.z}, {mx.x, mn.y, mx.z},
            {mx.x, mx.y, mx.z}, {mn.x, mx.y, mx.z}
        }};

        for (auto& p : pts) {
            out << "v " << p.x << " " << p.y << " " << p.z << "\n";
        }

        int face[12][3] = {
            {1,2,3},{1,3,4},{5,7,6},{5,8,7},
            {1,4,8},{1,8,5},{2,6,7},{2,7,3},
            {1,5,6},{1,6,2},{4,3,7},{4,7,8}
        };

        for (auto& f : face) {
            out << "f "
                << offset + f[0] << " "
                << offset + f[1] << " "
                << offset + f[2] << "\n";
        }

        offset += 8;
        totalVertices += 8;
        totalFaces += 12;
        return;
    }
```

```
    for (auto child : node->children) {
        writeVoxels(child, out, offset);
    }
}
```

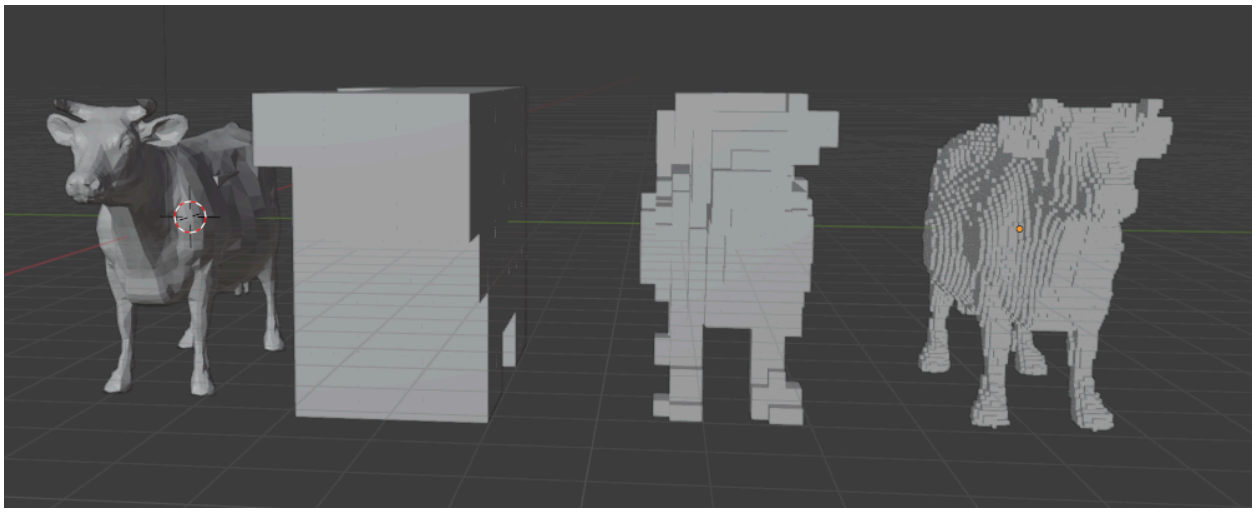
n. **Fungsi freeOctree**

Fungsi ini digunakan untuk menghapus seluruh node octree secara rekursif agar memori yang dialokasikan dengan new tidak bocor.

```
void freeOctree(nodeOctree* node) {
    if (!node) return;
    for (int i = 0; i < 8; i++) freeOctree(node->children[i]);
    delete node;
}
```

BAB III
PENGUJIAN

1. **Cow.obj dengan Depth 3, 5, 7**



Depth 3:

========== HASIL ==========
Jumlah voxel : 90
Jumlah vertex : 720
Jumlah faces : 1080
Kedalaman octree : 3
Waktu eksekusi : 0.021 detik
Path file output : test/cow-voxelized.obj

Depth 5:
========== HASIL ==========
Jumlah voxel : 1467
Jumlah vertex : 11736
Jumlah faces : 17604
Kedalaman octree : 5
Waktu eksekusi : 0.045 detik
Path file output : test/cow-voxelized.obj

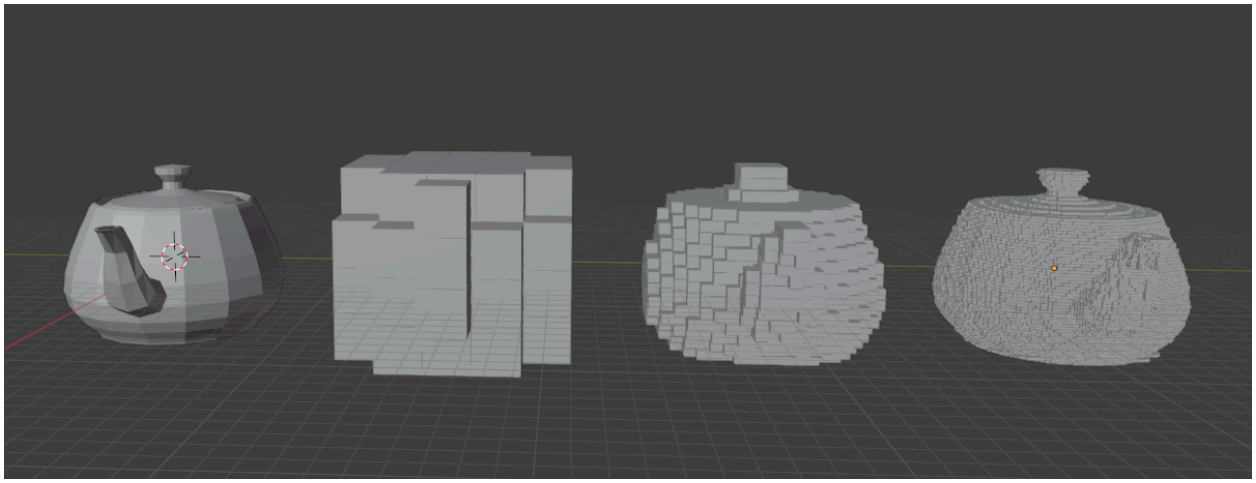
Depth 7:
========== HASIL ==========
Jumlah voxel : 25650
Jumlah vertex : 205200

Jumlah faces : 307800
Kedalaman octree : 7
Waktu eksekusi : 0.666 detik
Path file output : test/cow-voxelized.obj

--- Statistik node per kedalaman ---
0 : 1
1 : 8
2 : 32
3 : 160
4 : 720
5 : 2880
6 : 11736
7 : 49408

--- Node yang tidak perlu ditelusuri per kedalaman ---
1 : 4
2 : 12
3 : 70
4 : 360
5 : 1413
6 : 5560
7 : 23758

2. **Teapot.obj dengan depth 3,5,7**



Membaca file: test/teapot.obj
Vertex input: 530
Triangle input: 1024

Depth 3:

========== HASIL ==========
Jumlah voxel : 100
Jumlah vertex : 800
Jumlah faces : 1200
Kedalaman octree : 3
Waktu eksekusi : 0.008 detik
Path file output : test/teapot-voxelized.obj

Depth 5:
========== HASIL ==========
Jumlah voxel : 1859
Jumlah vertex : 14872
Jumlah faces : 22308
Kedalaman octree : 5
Waktu eksekusi : 0.04 detik
Path file output : test/teapot-voxelized.obj

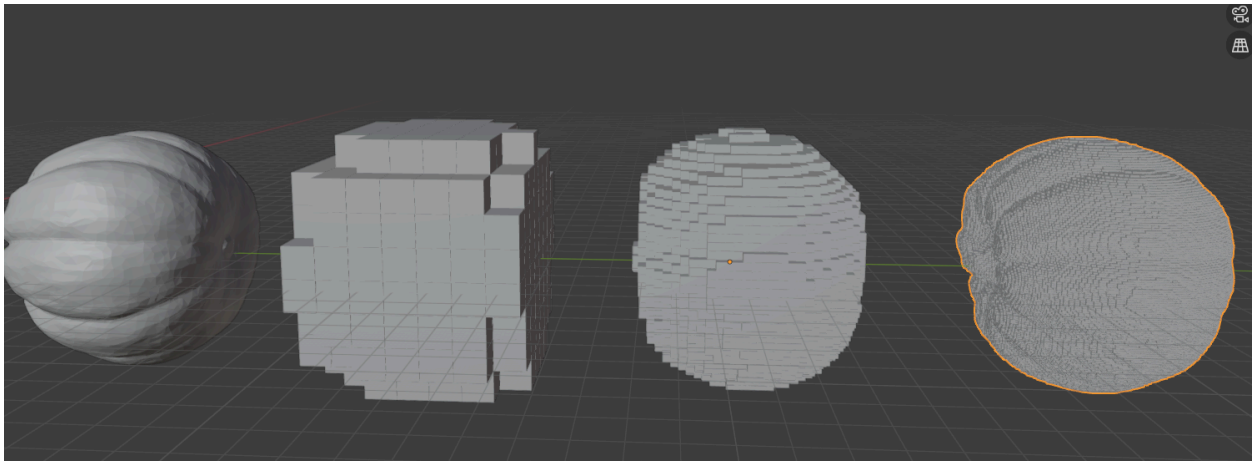
Depth 7:
========== HASIL ==========
Jumlah voxel : 31674
Jumlah vertex : 253392
Jumlah faces : 380088
Kedalaman octree : 7
Waktu eksekusi : 0.562 detik
Path file output : test/teapot-voxelized.obj

--- Statistik node per kedalaman ---
0 : 1
1 : 8
2 : 32
3 : 168
4 : 800
5 : 3560
6 : 14872
7 : 62328

--- Node yang tidak perlu ditelusuri per kedalaman ---
1 : 4
2 : 11
3 : 68
4 : 355

5 : 1701
6 : 7081
7 : 30654

3. **Pumpkin.obj dengan depth 3,5,7**



Membaca file: test/pumpkin.obj
Vertex input: 5002
Triangle input: 10000

Depth 3:
========== HASIL ==========
Jumlah voxel : 260
Jumlah vertex : 2080
Jumlah faces : 3120
Kedalaman octree : 3
Waktu eksekusi : 0.03 detik
Path file output : test/pumpkin-voxelized.obj

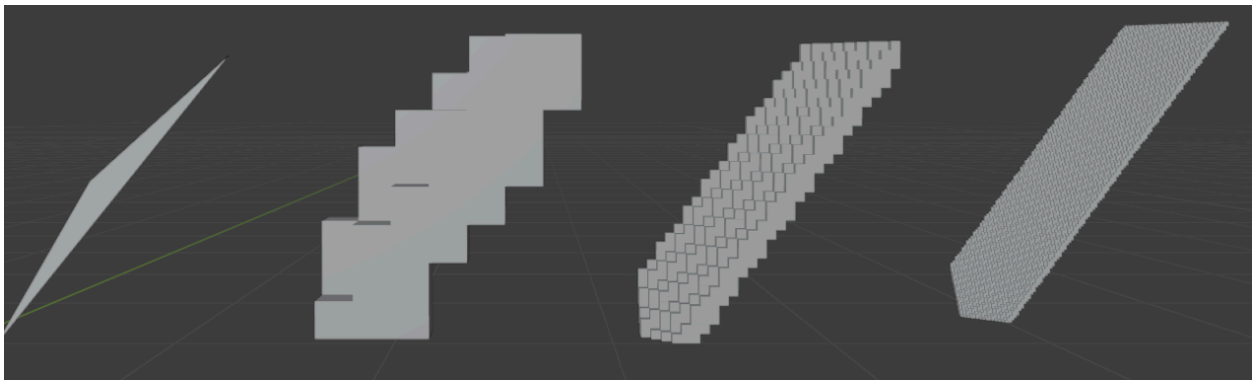
Depth 5:
========== HASIL ==========
Jumlah voxel : 4851
Jumlah vertex : 38808
Jumlah faces : 58212
Kedalaman octree : 5
Waktu eksekusi : 0.159 detik
Path file output : test/pumpkin-voxelized.obj

Depth 7:
========== HASIL ==========
Jumlah voxel : 79711
Jumlah vertex : 637688
Jumlah faces : 956532
Kedalaman octree : 7
Waktu eksekusi : 1.756 detik
Path file output : test/pumpkin-voxelized.obj

--- Statistik node per kedalaman ---
0 : 1
1 : 8
2 : 64
3 : 456
4 : 2080
5 : 9288
6 : 38808
7 : 157696

--- Node yang tidak perlu ditelusuri per kedalaman ---
2 : 7
3 : 196
4 : 919
5 : 4437
6 : 19096
7 : 77985

4. **Line.obj dengan depth 3,5,7**



Membaca file: test/line.obj
Vertex input: 3
Triangle input: 1

Depth 3:

========== HASIL ==========
Jumlah voxel : 44
Jumlah vertex : 352
Jumlah faces : 528
Kedalaman octree : 3
Waktu eksekusi : 0.006 detik
Path file output : test/line-voxelized.obj

Depth 5:

========== HASIL ==========
Jumlah voxel : 633
Jumlah vertex : 5064
Jumlah faces : 7596
Kedalaman octree : 5
Waktu eksekusi : 0.021 detik
Path file output : test/line-voxelized.obj

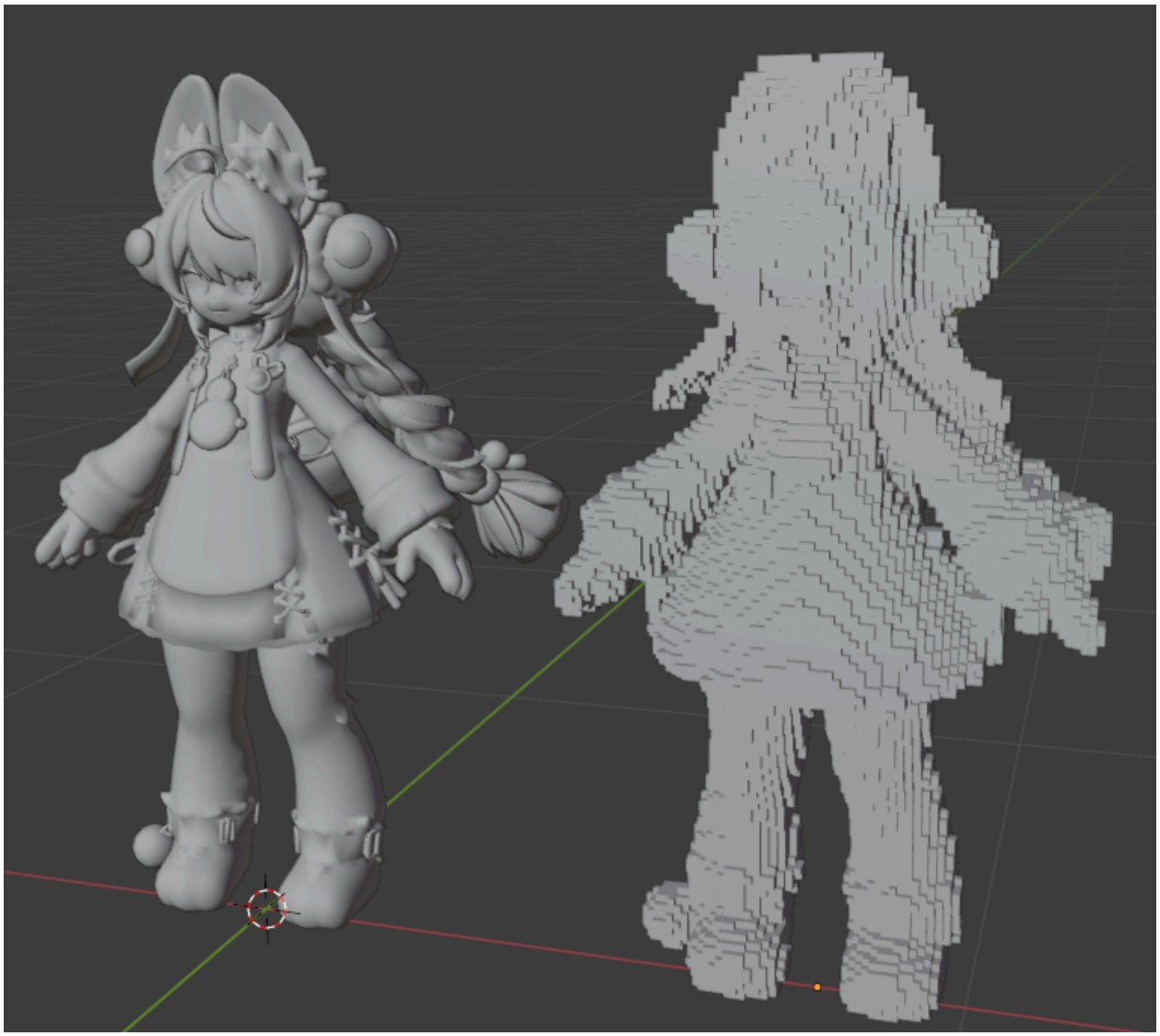
Depth 7:
========== HASIL ==========
Jumlah voxel : 9061
Jumlah vertex : 72488
Jumlah faces : 108732
Kedalaman octree : 7
Waktu eksekusi : 0.208 detik
Path file output : test/line-voxelized.obj

--- Statistik node per kedalaman ---
0 : 1
1 : 8
2 : 32
3 : 88
4 : 352
5 : 1448
6 : 5064
7 : 18856

--- Node yang tidak perlu ditelusuri per kedalaman ---
1 : 4
2 : 21
3 : 44
4 : 171

5 : 815
6 : 2707
7 : 9795

5. Zhao dengan depth 7



Membaca file: test/zhaos.obj
Vertex input: 38453
Triangle input: 56590

========== HASIL ==========
Jumlah voxel : 47664
Jumlah vertex : 381312

Jumlah faces : 571968
Kedalaman octree : 7
Waktu eksekusi : 1.209 detik
Path file output : test/zhaos-voxelized.obj

--- Statistik node per kedalaman ---
0 : 1
1 : 8
2 : 40
3 : 152
4 : 720
5 : 3480
6 : 16672
7 : 80816

--- Node yang tidak perlu ditelusuri per kedalaman ---
1 : 3
2 : 21
3 : 62
4 : 285
5 : 1396
6 : 6570
7 : 33152

BAB IV
PENUTUP

Kesimpulan

Dalam proyek tugas ini, algoritma Divide and Conquer dengan struktur data Octree berhasil digunakan untuk melakukan voxelisasi model 3D. Proses dimulai dari pembentukan bounding box awal yang melingkupi seluruh model, kemudian ruang dibagi secara rekursif menjadi delapan octant hingga mencapai kedalaman maksimum atau kondisi terminasi lainnya.

Kami berhasil mengimplementasikan proses pembagian ruang, penyaringan triangle pada setiap octant, serta pembentukan voxel permukaan berdasarkan node leaf hasil rekursi. Setiap voxel yang dihasilkan kemudian dikonversi menjadi representasi mesh 3D dan dituliskan ke file output dalam format OBJ.

Tautan

https://github.com/Fibarrr/Tucil2_13524125_13524133

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Daffa Arrizki Yanma

Muhammad Rafi Akbar

LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*		✓
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓